WEB4: A Technical Introduction

The trust-native internet, explained through its canonical equation

Dennis Palatov, GPT4o, Deepseek, Grok, Claude, Gemini, Manus

July 2026

Why Web4

AI agents now make purchases, execute code, and take decisions on behalf of the humans and organizations they serve. The internet they operate on has no native way to answer the two questions this raises:

1. **How do I know an agent will act appropriately in a given context, *before* it acts?** Today's answers: trust the platform that hosts it (Web2), or trust whoever holds the keys (Web3). Neither says anything about behavioral capability or contextual fit.
2. **How do I prove what an agent actually did, *after* the fact, without depending on a single trusted intermediary?** Today's answers: platform logs (revocable, manipulable) or blockchain records (limited expressivity, largely blind to off-chain action).

These are not future problems. They are the current, unsolved problems of agent delegation, agent-tool authorization, and any system where multiple agents — human or AI — must coordinate without a single referee.

The web arrived in layers, each solving the problem the previous one left open. **Web1** gave us access. **Web2** gave us participation, at the price of platform monopoly. **Web3** gave us ownership, at the price of token speculation. The problem now open is **trust between diverse intelligences** — and Web4's wager is that solving it requires trust to be a **first-class primitive of the protocol layer**: earned through witnessed interaction, expressed as verifiable structure, and inspectable by anyone. Not granted by a platform. Not purchased with a token.

The mechanism for that wager is **verifiable presence**. If every participating entity — human, AI, organization, role, task, or device — has a cryptographically anchored, non-transferable footprint that accumulates witnessed history, then trust can be *computed from the record* rather than *declared by an authority*. Everything in Web4 builds from that single move.

This paper is a technical introduction to the Web4 standard. It is deliberately scoped: it explains the **concepts** — what each mechanism is, why it exists, and how the pieces fit — and leaves specifications, schemas, and code to the normative standard it describes. The entire architecture can be stated in one line, and that line is where we begin.

Contents

WEB4: A Technical Introduction **1**
Why Web4 . . . 1

The Canonical Equation **4**

MCP: The I/O Membrane **5**
Why a membrane, not a pipe . . . 5
What Web4 adds to MCP . . . 5

RDF: The Ontological Backbone **5**
Why an ontology and not a database . . . 6
Where the backbone shows up . . . 6

LCT: The Presence Substrate **7**
What can be present . . . 7
The anatomy: what an LCT contains . . . 7
Birth: witnessed, or bootstrapped . . . 8
The capability ladder . . . 8
One presence, many devices . . . 9
Core properties . . . 9
Lifecycle: rotation, revocation, and a record that outlives participation . . . 9
Why this is the foundation . . . 10

T3/V3: The Trust and Value Tensors **10**
T3 — the Trust Tensor . . . 10
V3 — the Value Tensor . . . 10
Bound to entity-role pairs, in the graph . . . 10
Fractal, not fixed: the open-ended sub-graph . . . 11
How the numbers move . . . 11

MRH: The Markov Relevancy Horizon **12**
The idea . . . 12
The three canonical relationships . . . 12
Trust propagates along the edges . . . 13
Why the horizon is load-bearing . . . 13

ATP/ADP: The Value Cycle **13**
The cycle . . . 14
Why a metabolism, not a market . . . 14
Feedback, not foundation — and maturity, honestly . . . 15

Built on the Foundation **15**
The R6/R7 action grammar . . . 15
Roles as first-class entities . . . 16
Societies, authority, and law (SAL) . . . 16

The Standard and Current Implementations **17**

The standard . . . 17
Reference implementations . . . 17
Ecosystem bridges . . . 19

Conclusion **19**
Legal and organizational framework . . . 19
An invitation . . . 20

Glossary **20**
The equation elements . . . 20
Composed concepts . . . 21

References **22**
Primary Sources . . . 22
Patents . . . 22
Technical Implementations . . . 22
Related Work . . . 22
Theoretical Foundations . . . 22
Blockchain and Distributed Systems . . . 23
Memory and Cognition . . . 23
Trust and Reputation Systems . . . 23
Complex Systems and Emergence . . . 23
Collaborative Intelligence . . . 23
Web Evolution . . . 23
Additional Resources . . . 24
Websites . . . 24
Contact . . . 24
Contributing . . . 24

The Canonical Equation

The whole of Web4 is expressed in one equation:

```
Web4 = MCP + RDF + LCT + T3/V3 * MRH + ATP/ADP
```

with three operators, each carrying precise meaning:

Operator	Reading
`+`	"augmented with"
`*`	"contextualized by"
`/`	"verified by"

So the equation reads: Web4 is the Model Context Protocol, augmented with an RDF ontological backbone, augmented with Linked Context Tokens, augmented with trust tensors that are *verified by* value tensors and *contextualized by* each entity's Markov Relevancy Horizon, augmented with an ATP/ADP value cycle.

The equation is not a parts list — it encodes **structural relationships**. Web4 is an *ontology*: a formal structure of typed relationships, not a bundle of features. Each term earns its place by answering one question a trust-native internet must answer:

Term	Component	The question it answers
MCP	Model Context Protocol	How do entities *talk*? (the I/O membrane)
RDF	Resource Description Framework	How do statements *mean* anything? (the ontological backbone)
LCT	Linked Context Token	Who is *present*? (the presence substrate)
T3/V3	Trust / Value tensors	What can they be *trusted* to do, and what is their work *worth*?
MRH	Markov Relevancy Horizon	Within what *context* does any of this apply?
ATP/ADP	Allocation Transfer / Discharge Packets	How does *value* flow back to contribution?

Two structural facts in the equation deserve emphasis, because everything later depends on them:

Trust is contextualized, never absolute. The term is `T3/V3 * MRH` — the tensors never appear alone. There is no global reputation score in Web4; every trust assertion is bound to an entity *in a role, within a horizon*. A surgeon's trust does not follow her into a courtroom.

Value is feedback, not foundation. ATP/ADP is the *last* term. The value cycle presupposes presence (someone did the work), trust (its quality is assessable), and context (within some scope) — it rides on the foundation; it does not hold the foundation up.

The next six sections walk the equation left to right — one term at a time, in the order the architecture actually composes. Everything else in Web4 (actions, roles, societies, law, semantic bridging) is built *from* these six primitives, and is covered after them.

The equation's authoritative definition lives in the standard's glossary; its normative embedding is in the MCP protocol specification.

MCP: The I/O Membrane

The question it answers: how do entities talk?

Web4 does not invent a new wire protocol. It builds on the **Model Context Protocol (MCP)** — the emerging open standard through which AI models discover and invoke tools, read resources, and exchange context with the systems around them. MCP is where an agent's inside meets the world's outside, which is exactly where a trust architecture must live: **the membrane**.

Why a membrane, not a pipe

A pipe moves bytes. A membrane is selective — it knows what is inside, what is outside, and what may cross. Web4 adopts MCP as its I/O layer and then makes the membrane *trust-aware*:

- **Every crossing is attributable.** A tool call, a resource read, a context exchange — each is performed by an entity with verifiable presence (an LCT, introduced two sections from now), not by an anonymous connection.
- **Every crossing is contextual.** What an entity may do through the membrane depends on its role and its relevancy horizon, not merely on possession of a network path or an API key. Reachability is not authorization.
- **Crossings compose across boundaries.** When two Web4 *societies* (governed groups of entities, covered later) interact, they do so through MCP — the same membrane pattern, fractally repeated at every scale from a single agent's tool call to inter-society federation.

What Web4 adds to MCP

Plain MCP answers "how do I call this tool?" Web4's MCP profile adds the trust questions: *who* is calling (LCT-bound identity rather than a bearer credential), *in what capacity* (role), *within what scope* (MRH), and *on what record* (the call and its result become witnessed history that feeds trust). The ACP framework, covered later, extends this from *responding* agents to agents that *initiate* — with plans, approvals, and accountability.

The membrane also faces *inward*. The presence protocol specifies the MCP wire between an orchestrator and its own presence layer — a software vault (Hestia) or a hardware vault (Hardbound) exposing the same eight tools, from `connect` (which mints the session's *soft LCT*, giving even ephemeral sessions attributable presence) through `begin_action` / `record_outcome` (the R6 lifecycle at agent scale) to `request_witness`. The same grammar that governs societies governs a single agent's afternoon — the membrane pattern applied to the boundary between deciding and doing.

The design principle carried throughout: **cooperation flows through the membrane; the membrane never relies on cooperation.** Structured, low-friction channels make the compliant path the easy path, while enforcement stays at the boundary itself.

Normative reference: `core-spec/mcp-protocol.md`.

RDF: The Ontological Backbone

The question it answers: how do statements mean anything?

A trust-native internet is, above all, a system of *statements*: "this agent is bound to this hardware," "this role was performed with this competence," "this act was witnessed by these entities." For those statements to be verifiable, they must first be **machine-readable, typed, and composable**. Web4 expresses every relationship as an **RDF triple** — subject, typed predicate, object — using the W3C Resource Description Framework.

Why an ontology and not a database

The difference matters more than it first appears. A protocol defines message formats; a database holds rows; an **ontology defines what things mean and how they relate**. When Web4 says "trust," it does not mean a number in someone's table — it means a typed relationship in a graph that any RDF-speaking system can query, extend, and reason over.

This is not a metaphor layered over a conventional store; the vocabulary is concrete and published. Web4's statements live under two namespaces — `web4:` for the ontology, `lct:` for presence instances — and the canonical relationships are real predicates with defined semantics:

```
@prefix web4: <https://web4.io/ontology#> .
@prefix lct:  <https://web4.io/lct/> .

lct:alice   web4:boundTo      lct:device-anchor-1 .   # permanent hierarchical attachment
lct:bob     web4:pairedWith   lct:surgeon-role .      # authorized operational connection
lct:carol   web4:witnessedBy  lct:audit-oracle-7 .    # trust through observation
```

Each root predicate fans out into sub-properties — `web4:parentBinding`, `web4:energyPairing`, `web4:timeWitness`, and their siblings, all `rdfs:subPropertyOf web4:hasRelationship` — so a query can ask the coarse question ("is there *any* relationship?") or the precise one ("is there a *witnessing* relationship of the *audit* kind?") against the same graph. Edge metadata (what kind of binding, since when, how many times observed) rides on the triples through RDF reification, so the graph carries not just *that* two entities relate, but the witnessed terms on which they do.

Three properties follow directly:

- **Extensibility without central coordination.** Anyone can add a new trust sub-dimension, a new relationship type, or a new witness kind by adding vocabulary — no core-protocol change, no permission from a registry. This is what keeps the standard small while the ecosystem grows. The pattern that makes this safe is `web4:subDimensionOf`: new terms declare what they refine, so extensions slot *into* the existing graph instead of forking it (the T3/V3 section uses this to make trust itself open-ended).
- **Semantic interoperability.** Web4 statements compose with the existing semantic-web world (W3C vocabularies, SPARQL, linked data) rather than creating another silo. A relevancy query is a SPARQL traversal with a depth bound, not a proprietary API call.
- **Fractal structure.** The same triple pattern describes a sensor reading, a role assignment, and an inter-society treaty. Meaning scales without changing shape.

Where the backbone shows up

Every other term in the equation is *realized* in RDF. The LCT itself is a JSON-LD node (`@type: web4:LinkedContextToken`) whose identifier resolves into the graph. Its MRH — the relevancy horizon — is exactly the subgraph of typed `boundTo` / `pairedWith` / `witnessedBy` edges rooted at that node, traversable within a bounded depth. Trust and value tensors are RDF sub-graphs

rooted at three dimensions each (not fixed vectors), with scores attached as witnessed, timestamped statements. The backbone is why the equation's components interlock instead of merely coexisting: they are all the same graph, viewed from different terms.

Normative reference: the ontology artifacts at web4-standard/ontology/ (Turtle + JSON-LD), with the relationship vocabulary in core-spec/mrh-tensors.md and the graph model in MRH_RDF_SPECIFICATION.md.

LCT: The Presence Substrate

The question it answers: who is present?

The **Linked Context Token (LCT)** is Web4's foundational primitive: a non-transferable, cryptographically anchored record permanently bound to exactly one entity for the duration of its participation. It is not an account, not a wallet, and not merely an identifier — it is the **reification of presence itself**. An LCT crystallizes when an entity enters Web4 and accompanies it until its participation ends.

The standard puts the distinction in one sentence: *an LCT is not an identity; it is a presence.* An identity answers "who are you?" A presence answers "who is here, acting, accumulating a record?" — and it is the record, not the claim, that everything else in Web4 computes on.

What can be present

Web4 deliberately widens what counts as an entity — anything that can leave a footprint can have one:

- **Humans** and **AI agents** (the obvious participants)
- **Organizations** (collective entities with their own presence)
- **Roles** (a job itself, distinct from whoever performs it — a key move covered later)
- **Tasks and resources** (things that exist, execute, complete, or get consumed)
- **Devices** (hardware that senses and acts)

The anatomy: what an LCT contains

An LCT is a JSON-LD document (`@type: web4:LinkedContextToken`) with **six required components**, and the list is the single most structural fact in this paper:

1. **Identity** — the `lct_id` and the subject's DID.
2. **Binding** — the cryptographic anchor and its `binding_proof`.
3. **MRH** — the entity's Markov Relevancy Horizon: its typed relationship graph.
4. **Policy** — the constraints under which the presence acts.
5. **T3** — the trust tensor (talent / training / temperament).
6. **V3** — the value tensor (valuation / veracity / validity).

Read that list against the equation. The `LCT` term and the `T3/V3 * MRH` term are not separate systems that reference each other — **the token is the container**: every LCT *carries* its own relevancy horizon and its own trust and value tensors as mandatory fields. The MRH section and the tensor section that follow describe the contents of every LCT ever minted, not auxiliary databases beside it.

The identifier is **derived from the binding key**. The binding (entity, key, optional hardware anchor) is serialized as deterministic CBOR and signed as a COSE_Sign1 `binding_proof`; the LCT's identifier is a pure function of the binding public key: `lct:web4:mb32:b` + base32(sha256(raw public key)), at full digest width. No registry allocates it, so an LCT can be minted offline or in a society the verifier has never met, and its uniqueness rests on key-generation entropy, not on coordination. Any registry or verifier re-derives the identifier from the document's own key and rejects a mismatch, so an identifier cannot be pointed at a key it does not belong to. That check proves consistency, not possession: *acting* as the identifier still requires a valid `binding_proof` signed by the key. Because the identifier follows the key, a key rotation yields a new identifier, and continuity is carried by lineage (below). What the key makes unique is the *identifier*, not the presence behind it. A copied key or seed gives two presences one identifier, and nothing in the identifier can tell them apart. Which one is the original is a question of evidence, and that evidence is the MRH: relationships that *counterparties* signed under their own keys. A copy can carry the existing signed records byte for byte, and it may go on to collect new signatures, but holding the key does not let it make the counterparties' signatures itself. What can tell two holders apart is divergence (relationships one holds that conflict with the other's, or acts witnessed from two places at once) and continuity with the counterparties of the original's witnessed lineage, weighed under the society's law; a copy that stays passive or mirrors the original may show neither. The standard classifies every way two things can appear to share an identity (re-mint, binding failure, concurrent presence from a copied seed, digest collision, an unwitnessed foreign presence) and gives each an adjudication path under the society's law. Where the evidence cannot tell the holders apart, the outcome is recorded as unresolved, not guessed.

Birth: witnessed, or bootstrapped

The primary issuance path is a **society-issued birth certificate**. An entity requests presence; its society validates citizenship; a binding ceremony runs with a quorum of **at least three birth witnesses**; and the new LCT's MRH is initialized from the ceremony itself — the witnesses become its first `witnessing` edges, the citizen role becomes a permanent `paired` edge, the hardware anchor becomes a `bound` edge. Presence begins *already woven into* the witnessing fabric, with initial T3/V3 computed and the token published to the society's registry.

A bootstrap path exists for entities without a society: the **self-issued LCT** — self-signed binding proof, empty MRH, minimal trust. It is real presence, but presence that has corroborated nothing yet. The distance between the two paths is exactly the distance trust must travel, and the capability ladder below makes that distance legible rather than binary.

The capability ladder

LCTs are graded into six **capability levels** (0–5), each with explicit required fields: **STUB** (a placeholder), **MINIMAL** (self-issued), **BASIC** (at least one MRH relationship), **STANDARD** (witnessed, with oracle-computed tensors and attestation), **FULL** (birth certificate, $\geq$3 witnesses, permanent citizen pairing), and **HARDWARE** (a hardware-anchored EAT attestation). Levels 0$\rightarrow$4 are additive — an LCT accretes capability as its record grows. Level 4$\rightarrow$5 is **re-issuance**: hardware binding cannot be bolted on after the fact, because the anchor must sit inside the signed binding from creation. Unimplemented components are carried as explicit stubs (`{stub: true, reason}`) — partial presence is declared, not hidden.

One presence, many devices

A modern entity acts through several devices, and Web4's answer inverts the usual instinct that more endpoints mean more attack surface. A **Root LCT** carries a *device constellation*; each device holds its own **Device LCT** bound to one hardware anchor (phone secure enclave, FIDO2 key, TPM, or software fallback), cross-witnessing the others. Trust is **capped by anchor composition**: a single software key tops out at 0.40, a phone enclave or TPM alone at 0.75, and three or more diverse hardware anchors at 0.98. Recovery follows a quorum of the constellation itself. The design principle: **identity is coherence across witnesses** — a presence that many independent anchors keep agreeing on is *stronger*, not weaker, for each device added.

Core properties

Permanently bound and non-transferable. An LCT cannot be sold, given away, or moved between entities. This is not a limitation but the structural guarantee that makes reputation *mean* something: every witnessed interaction traces back to the entity that actually participated. Trust histories cannot be bought, inherited, or laundered.

Cryptographically anchored. Each LCT roots in keypair-backed identity, with an optional hardware-binding ladder (from software keys up to TPM- and secure-enclave-anchored attestation). "I am the entity bound to this LCT, here is a fresh signature over your challenge" is a claim cryptography can check — unlike "I am @alice," which is a claim a platform accepts.

Witness-hardened. An LCT's strength is not secrecy but *accumulated corroboration*. Every interaction can be witnessed by other entities; every independent witness link makes the presence harder to forge and its history harder to dispute. Trust in Web4 is built from this witnessing fabric — presence that has been repeatedly, independently observed. Corroboration is weighed, not counted. A signature proves that a key signed, not that its holder is independent of the subject, so a relying party weighs each witness by its own standing and by its independence from the others. A large record built from mutually dependent signers can be worth less than a small one built from independent, anchored ones.

Linked and contextual. LCTs form malleable links to other LCTs — trust webs, delegation chains, parent/child lineage. The token is permanent; its *expression* is contextual (the same doctor's presence carries different weight in a medical forum than a book club — a fact made precise by the MRH, two sections ahead, which lives inside the token itself).

Lifecycle: rotation, revocation, and a record that outlives participation

An LCT is created (genesis), lives (active), and can **rotate**: keys change by issuing a successor LCT under the same subject DID, with explicit lineage to the parent and a 24–48 hour overlap before the parent is marked superseded. It can be **revoked** — for compromise, supersession, expiry, or violation — which disables its capabilities while preserving its MRH read-only: the graph of what it touched remains queryable evidence. Participation ends in one of two states — *void* for a natural ending, *slashed* for a trust violation — and in both cases **the record persists beyond conclusion**. Accountability outlives participation; that is the point of presence rather than identity.

Underneath the entire mechanism runs one discipline the standard states explicitly: **inspectable evidence, not prescribed trust.** The protocol's job is to make the evidence unforgeable — the binding, the witnesses, the history. Whether to trust, and for what, remains the relying party's contextual call.

Why this is the foundation

Every subsequent term in the equation presupposes this one. Trust tensors describe *an LCT's* capability — and live inside it. Relevancy horizons scope *an LCT's* context — and live inside it. Value cycles reward *an LCT's* contribution. Verifiable presence is the move that converts "trust as declaration" into "trust as computable record" — the rest of the architecture is the machinery that computes it.

Normative reference: `core-spec/LCT-linked-context-token.md` (Core Specification v1.0.0), with `lct-capability-levels.md` for the capability ladder, `multi-device-lct-binding.md` for device constellations, `entity-types.md` for the entity taxonomy, and `did-web4-method.md` for the DID-standard bridge.

T3/V3: The Trust and Value Tensors

The question they answer: what can an entity be trusted to do — and what is its work worth?

Presence alone says *who is here*, not *what they're good for*. Web4 measures capability and contribution with two three-dimensional tensors, each dimension itself extensible into finer structure. Both tensors are **required components of every LCT** — they ship inside the presence token, not in a separate reputation service — and, as this section shows, the same tensors are simultaneously *bound into the graph* at entity-role pairs. That dual residence is the design's center of gravity.

T3 — the Trust Tensor

T3 captures *capability*: what an entity can be trusted to do, along three root dimensions:

- **Talent** — inherent aptitude for the kind of work
- **Training** — acquired knowledge and demonstrated skill
- **Temperament** — behavioral reliability: consistency, judgment, restraint

V3 — the Value Tensor

V3 captures *contribution*: what an entity's output is worth, along three root dimensions:

- **Valuation** — the worth ascribed by those who received the value
- **Veracity** — objective accuracy and reproducibility of what was delivered
- **Validity** — confirmed, witnessed transfer — the value actually arrived

The equation writes the pair as **T3/V3** — trust *verified by* value. Claimed capability is continuously checked against delivered contribution: an entity whose T3 says "expert" but whose V3 record shows little validated value will see the gap; sustained delivery closes it. Verification, not assertion.

Bound to entity-role pairs, in the graph

There is no "Alice's trust score." The standard is a hard invariant here: implementations **must not** compute global, role-agnostic trust. Tensors exist only within role contexts, and the binding is an RDF fact, not a convention:

```
_:tensor1 a web4:T3Tensor ;
    web4:entity      lct:alice ;
    web4:role        web4:Surgeon ;
    web4:talent      0.95 ;
    web4:training    0.92 ;
    web4:temperament 0.88 .
```

Alice-as-surgeon, Alice-as-reviewer, Alice-as-citizen each accumulate their own tensor from their own witnessed history. (V3 rides on the same pairing: the value tensor derives its entity-role context from the co-located T3 tensor, so "Alice-as-surgeon's delivered value" is the same node in the graph.) A new role starts at minimal trust; competence in one capacity is evidence of nothing in another, and cross-role transfer requires an explicit, inspectable bridge.

Fractal, not fixed: the open-ended sub-graph

Each root dimension is the root node of an **open-ended RDF sub-graph** of context-specific sub-dimensions, declared with the `web4:subDimensionOf` pattern:

```
analytics:StatisticalModeling a web4:Dimension ;
    web4:subDimensionOf web4:Talent .
analytics:BayesianInference a web4:Dimension ;
    web4:subDimensionOf analytics:StatisticalModeling .   # fractal depth
```

A medical society can refine *Training* into `surgical-technique` and `diagnostic-accuracy` without touching the standard — and so can any other domain, indefinitely deep. Individual scores attach as **witnessed, timestamped statements** (dimension, score, `observedAt`, `witnessedBy`), so every number in a tensor carries its provenance. The shorthand form (`web4:talent 0.85`) is the aggregate of the sub-graph rooted at that dimension; composite scores fold the three roots together with published weights (T3: 0.4/0.3/0.3; V3: 0.3/0.35/0.35).

How the numbers move

Tensor values change only on **witnessed outcomes**, through two normative paths. The categorical outcome table maps result classes to deltas — a novel success is worth +0.02–0.05 on the relevant capability; an ethics violation costs −0.05 talent and −0.10 temperament. The continuous path adjusts by delivered quality: `delta = 0.02 × (quality - 0.5)`, scaled per dimension (temperament moves slowest — reliability is demonstrated over time, not in a single act). All deltas clamp to [0,1].

Decay is asymmetric by design: unexercised **Training** erodes (−0.001/month), **Temperament** recovers slowly (+0.01/month — the path back from a violation is deliberately longer than the fall), and **Talent never decays** — inherent aptitude is not spent by disuse, and the standard pins this as a protocol invariant guarded by conformance vectors.

At the society scale, R7 transactions emit **reputation deltas** triggered by published Law Oracle rules — which action types, result statuses, and quality thresholds move which dimensions, by how much. Deltas are witnessed (selected from MRH-proximate entities), time-weighted onto a 0.5 neutral baseline with exponential decay, and deliberately **asymmetric**: violation costs are large relative to success gains, so reputation cannot be farmed to launder later coercion. In Web4, trust is not a side effect of the record — computing it *is* the product.

Two honest caveats. First, legacy flat six-dimension schemas from the project's early lineage are reconciled through a normative 6D→3D bridge rather than silently coexisting. Second, the range of V3 **Valuation** is a declared open question in the standard — the specification permits values above 1.0 (exceptional worth), the reference SDK currently clamps to [0,1], and the discrepancy is tracked as an operator decision rather than papered over.

Because the tensors route real decisions (who gets the role, whose output is accepted, where value flows), and because they are always role- and context-scoped, they resist the two classic reputation failures: the global score that follows you where it shouldn't, and the purchased reputation that was never earned.

Normative reference: `core-spec/t3-v3-tensors.md` and `core-spec/reputation-computation.md`, with the ontology at `ontology/t3v3-ontology.ttl`.

MRH: The Markov Relevancy Horizon

The question it answers: within what context does any of this apply?

Nothing in Web4 is global — not trust, not authorization, not attention. The **Markov Relevancy Horizon (MRH)** is each entity's zone of relevance: what it can perceive, act on, and be affected by. The equation's central term is `T3/V3 * MRH` — trust *contextualized by* horizon — and the `*` is doing the heaviest lifting in the whole equation.

The MRH is not an external structure consulted about an entity. As the LCT section established, it is a **required component inside every LCT** — each presence carries its own horizon, and the horizon travels with the token.

The idea

The name borrows deliberately from the Markov property: just as a Markov process's next state depends only on the present state — not the entire past — an entity's *relevant context* is bounded. Not everything everywhere matters to everyone. The MRH makes that boundary explicit, verifiable, and usable:

- What information should reach this entity?
- What actions can it meaningfully take?
- Which other entities fall within its sphere?
- Over what time horizon do its concerns extend?

The three canonical relationships

An MRH is a **typed RDF relationship graph**, and its edges come in exactly three canonical kinds — the same three predicates the RDF section introduced — each with its own trust semantics:

- **`web4:boundTo` — permanent hierarchical attachments.** Parent, child, and sibling bindings: the entity's own hardware anchors, its device constellation, its lineage. Trust flows bidirectionally and strongly — these are the edges an entity cannot disown without ceasing to be itself.
- **`web4:pairedWith` — authorized operational connections.** Pairings to roles and counterparties, created by explicit authorization. Birth-certificate pairings (the citizen role granted at issuance) are permanent; ordinary operational pairings need not be.

- `web4:witnessedBy` — **trust through observation.** The witnessing fabric made graph-shaped. Witness edges carry a *kind* — time, audit, oracle, existence, action, state, quality — plus a running count and a last-attestation timestamp. Trust flows one way: from the witness toward the witnessed, as accumulated corroboration.

Relevance is computed by **traversal**: what can be reached from this LCT's node, through which edge types, within a bounded number of hops. The bound has a default — `horizon_depth: 3` — and it is where the "Markov" in the name becomes literal: beyond the depth bound, relationships are *defined as irrelevant*. Not distrusted — out of scope.

Trust propagates along the edges

The horizon is not only a filter; it is a medium. Confidence in a distant entity is computed over the paths that reach it, and the standard defines three propagation algorithms a relying party can choose among: **multiplicative** (path trust decays hop by hop, default factor 0.7 — a direct witness outweighs a friend of a friend of a friend), **probabilistic** (independent paths combine noisy-OR style: `1 - ∏(1 - path_trust)` — corroboration along genuinely independent routes accumulates), and **maximal** (the best single path governs). Because paths are graph objects, provenance is inspectable: two paths that secretly share an origin can be discounted as one.

The graph is **dynamic by obligation**, not by convention: the standard requires the MRH to be updated on every new binding, pairing, witness event, revocation, and trust recomputation. Horizons grow and shrink with demonstrated behavior — a new agent starts narrow and earns reach; a misbehaving one contracts.

This gives the horizon three properties a flat boundary cannot have:

- **Asymmetry.** A can be within B's horizon while B is outside A's — delegation and witnessing are directional.
- **Multi-path trust.** Confidence in a distant entity can accumulate along independent graph paths, and be discounted where paths share provenance.
- **Dynamism.** Reach is earned and lost on the record, in both directions.

Why the horizon is load-bearing

Two consequences make MRH more than bookkeeping. First, **contextualized trust becomes enforceable**. Role-scoped trust queries evaluate *inside* the graph: ask for an entity's trust as a surgeon outside the medical horizon and the defined answer is not "low" — it is 0.0, out of context. `T3/V3 * MRH` means the surgeon's tensor simply *does not apply* elsewhere; misapplied reputation is a type error, not a policy violation. Second, **reachability stops implying authorization**: an entity may be network-reachable and still be outside the horizon for an action. The same scoping disciplines the trust machinery itself — reputation witnesses, for instance, are drawn from MRH-proximate entities rather than the open network. In an internet of autonomous agents, that inversion — context as the gate, not connectivity — is the security model.

Normative reference: `core-spec/mrh-tensors.md`, with the graph model in `MRH_RDF_SPECIFICATION.md` and the MRH's residence inside the token in `core-spec/LCT-linked-context-token.md`.

ATP/ADP: The Value Cycle

The question it answers: how does value flow back to contribution?

The final term of the equation closes the loop. With presence established (LCT), capability measured (T3/V3), and context bounded (MRH), one question remains: how does the system *allocate* — energy, attention, compute, resources — so that contribution is rewarded and waste is not? Web4's answer is the **ATP/ADP cycle**, named for the molecule that carries energy in every living cell.

The cycle

Allocation Transfer Packets exist in two states, forever cycling:

- **ATP (charged)** — allocation ready to fuel work
- **ADP (Allocation Discharge Packet)** — allocation spent, carrying the record of what it was spent on, awaiting recognition

Work *discharges* ATP into ADP. Witnessed, recognized contribution *recharges* ADP back into ATP. The tokens are **semi-fungible**: units are equivalent as allocation, but each carries its history — what was attempted, by whom, to what result — context that matters when value is assessed.

The mechanics are deliberately conservative. Societies **mint** the supply (in the discharged ADP state) and hold it in governed pools; the cycle obeys a strict **conservation invariant** — total supply always equals charged plus discharged, and every transfer balances to the unit, fees recycled to the pool rather than destroyed. The one sanctioned exception is **slashing**: witnessed, evidence-backed destruction for violations — the economic analogue of the tensor's asymmetric accrual, and equally deliberate.

Two rules weld the cycle into everything that came before it. First, **discharge happens only through the R6 action grammar**: spending allocation *is* transacting — every R6 Request carries an `atpStake`, locked in escrow at validation and settled atomically with the action's costs, so allocation can never move without a governed, witnessed act attached. Second, **recharge requires proof**: charging ADP back to ATP demands producer authorization under society law plus a cryptographic value proof — recognition is not automatic. Whether discharged work recharges depends on the *receivers* of the value attesting it through the V3 lens (was it valuable? was it accurate? did it arrive?). That is the equation's structure made operational: the value cycle runs *through* the trust layer, not beside it.

And the weld runs both directions: every ATP event feeds the tensors — recognized charging raises Training and Valuation; slashing cuts Temperament and Veracity. An entity's economic record and its trust record are one record, viewed through two terms of the equation. (An LCT may even surface its allocation position as an `energy_balance` sub-dimension of V3 Valuation — capability level 3 expects it — so "does this entity do recognized work?" is a graph query, not an audit.)

Why a metabolism, not a market

The deliberate contrast is with mining and staking. Proof-of-work rewards burning energy on puzzles; proof-of-stake rewards already having tokens. Both decouple reward from *usefulness*. The ATP/ADP cycle couples them by construction:

- **You cannot accumulate allocation without contributing** — recharge requires witnessed, recognized value delivery. There is no "early holder" position to speculate from.
- **You cannot fake contribution** — the discharge record and its witnesses are part of the trust fabric; gaming attempts damage the T3/V3 tensors that gate future allocation.

- **Hoarding is self-limiting** — allocation that never discharges does no work and earns no recognition, and societies may levy **demurrage**: a maintenance discharge on idle charge, so dormant allocation slowly returns to the pool. The system favors flow over accumulation, as metabolisms do.

The design intent is sometimes summarized as *anti-Ponzi*: value in the system tracks work performed for identifiable beneficiaries, not the recruitment of later participants.

Feedback, not foundation — and maturity, honestly

Two clarifications this paper owes the reader. First, ATP/ADP is **the feedback layer, not the foundation**: it presupposes every prior term of the equation, and nothing in presence, trust, or context *depends on* it — which is why it is the last term, not the first. Second, its maturity is honestly mixed. The *mechanism* now ships in public: the account model and conservation invariants are implemented in the published `web4-core` crate and the Python SDK, validated against the standard's test vectors. What does not yet exist is a *live economy* — no deployment circulates real allocation at scale, and large-scale economic attack modeling remains open research. The accounting runs; the metabolism it is meant to govern is still ahead.

Normative reference: core-spec/atp-adp-cycle.md, with the discharge weld in core-spec/r6-framework.md.

Built on the Foundation

The six primitives of the equation are the alphabet. This section covers the words Web4 spells with them: how actions happen, how roles work, how groups govern themselves, and how meaning survives crossing domains. Each is specified in the standard; each is *composed from* the primitives rather than added beside them.

The R6/R7 action grammar

Every action in Web4 — from a tool call to a governance decision — has the same anatomy, named **R6**:

```
Rules + Role + Request + Reference + Resource → Result
```

- **Rules** — what governs the action: the society's law by hash (`lawHash`), plus constraints, permissions, and prohibitions
- **Role** — the capacity in which the actor acts: the actor's LCT paired to the role's LCT, carrying that pairing's own `t3InRole` / `v3InRole` and permissions
- **Request** — the explicit intent: the action verb, target, parameters, acceptance criteria, a nonce, the `atpStake` backing it, and — for delegated acts — a `proofOfAgency` over the grant
- **Reference** — the context brought to bear: precedent, interpretations, witnesses, and an `mrhContext` (relevant entities and trust paths within a depth bound)
- **Resource** — what the action consumes: required and available ATP, compute, bandwidth — with the stake locked in `escrow` under a release condition
- **Result** — what actually happened: status, output, resources consumed, tensor updates, attestations, and a ledger proof — signed and witnessed

A Request moves through the stack in three phases. **Pre-execution validation**: the role pairing is verified in the actor's own MRH, the agency grant and its scope are checked, the law in the

Rules slot is applied, resources are confirmed, and the escrow locks. **Execution**: metered, inside the role context, output validated against the rules. **Settlement**: atomic — costs computed, ATP transferred or escrow refunded, the role-pairing's T3/V3 updated, the ledger entry written with its witnesses, and the actor's MRH updated with the act. A Request rejected before execution is an `error` (never attempted); a Request that fails mid-execution is a `failure` (attempted, metered, settled) — the distinction is part of the record, because the two carry different trust meaning.

R7 extends the grammar with a seventh element as first-class *output*: **Reputation**. The delta between Request and Result feeds back into the actor's T3/V3 tensors — every action doesn't just produce an outcome, it *updates the trust record*. This is the mechanism behind the earlier claim that trust is computed rather than declared: R7 is where the computation happens, one action at a time.

The grammar makes action **auditable by shape**: any observer can ask of any act — under what rules? in which role? requested what? on what basis? at what cost? with what result, and what did it do to reputation?

Roles as first-class entities

In Web4, a role is not a label on a user — it is an **entity with its own LCT**. "Data Analyst" has presence: its own requirements, its own permission boundaries, and its own accumulated history of who has performed it and how well. When an agent takes on a role, their LCTs pair; performance updates both records — the agent's tensor *in that role*, and the role's own history.

This one move does a lot of work. It is why trust can be role-scoped (the tensor binds to the *pairing*), why delegation is inspectable (authority flows through explicit role links), and why capability markets become possible (roles can be discovered and matched on verifiable history rather than claimed credentials).

The standard also fixes a floor. Every Web4 society must have eight **base-mandatory roles** filled — Sovereign, Law Oracle, Policy-Entity, Treasurer, Administrator, Archivist, Citizen, and Maintainer — though one entity may hold several, down to a solo founder filling all eight. The **Maintainer** is worth singling out because it is the standard's cleanest statement of a distinction this paper has already drawn. Its job is to examine its entity's own standing state on a declared cadence — which members exist, which wiring is live, which documented assumption still holds, whether the resources the entity's commitments depend on are still there — and to record each finding with its evidence and a *clearing condition*, including the finding that nothing is wrong. It is defined for any entity with an MRH, not only for societies: a hub, a device, a service, or a role may each hold a Maintainer of its own. And it creates no authority. The entity's MRH bounds what the Maintainer may *consider*; the entity's law and whatever scope it has explicitly delegated bound what the Maintainer may *do*. A repair outside that scope is refused by the same gate that refuses any other unauthorized act, and the Maintainer's only recourse is to record the finding and escalate. Relevance is not permission — which is what makes the role safe to run unattended.

Societies, authority, and law (SAL)

Entities coordinate in **societies**: groups with membership, shared rules, and collective memory. The **Society–Authority–Law** pattern specifies how governance works without a central platform:

- **Society** — a bounded group of LCT-bearing members; an entity in its own right, with its own presence

- **Authority** — delegated, never assumed: specific powers flow to specific roles through explicit, revocable grants (the **AGY** delegation pattern), every grant an inspectable link in the graph
- **Law** — *data, not code comments*: the society's rules are published as inspectable, versioned artifacts that gate actions through the R6 grammar's **Rules** slot. Members can read the law that binds them; enforcement decisions cite the rule they applied

A society's record-keeping runs on **witnessing**: acts are recorded, hash-chained, and counter-signed by members, making the collective memory tamper-evident without requiring a global blockchain. Societies interact with other societies through the same MCP membrane individuals use — governance, like everything else in Web4, is fractal.

Two protocol layers complete the picture. **ACP** (Agentic Context Protocol) extends MCP for agents that *initiate* rather than merely respond — plan, seek approval, execute, record — keeping autonomous action inside the accountability grammar. And **dictionary entities** handle meaning across boundaries: living, LCT-bearing translators between domain vocabularies that accumulate their own trust records as they translate, so that semantic fidelity itself becomes witnessed and measurable ("compression-trust").

Normative references: *r6-framework.md* · *r7-framework.md* · *society-roles.md* · *web4-society-authority-law.md* · *acp-framework.md* · *dictionary-entities.md*.

The Standard and Current Implementations

Web4 is developed in the open as a **normative standard with reference implementations**. This section is the map from the concepts in this paper to the artifacts you can read, run, and challenge.

The standard

`web4-standard/` is the canonical specification tree:

- `core-spec/` — the normative documents: one spec per mechanism this paper introduced (LCT, T3/V3, MRH, ATP/ADP, MCP, R6/R7, SAL, ACP, dictionaries, entity types, the `did:web4` DID method, security framework, error taxonomy)
- `ontology/` — the RDF backbone as machine-readable artifacts (Turtle ontologies, JSON-LD contexts)
- `test-vectors/` and `profiles/` — conformance vectors by subsystem, and deployment profiles (edge device, cloud service, peer-to-peer)

Specification status is marked unevenly. Some documents carry an explicit status line — v1.0 core specification, draft under active revision, proposed standard — and many carry none at all, including the specs for several of the mechanisms listed above. Where a document is unmarked, read its status as unsettled rather than settled: absence of a status line is not a claim of stability. The standard is versioned in public; its history is its changelog.

Reference implementations

Concepts prove themselves by running. Three public codebases currently implement the standard, at different layers:

Core packages — the primitives as libraries. `web4-core` and `web4-trust-core` ship the LCT presence primitive, T3/V3 tensors, the ATP account model with its conservation invariants, the R6/R7 grammar, ledger backends, and the attestation envelope as installable packages. Both are published Rust crates on crates.io, and both ship Python wheels on PyPI — `web4-core` under its own name, the trust crate under the distribution name `web4-trust` rather than `web4-trust-core`. `web4-trust-core` additionally ships WASM browser bindings on npm. A third crate, `web4-policy`, is the SAL pattern's law engine as a library: a society's law as data — norms, procedures, and escalation triggers — evaluated against an R6 request to allow, deny, or escalate, with each domain supplying its own policy shape on top of the generic engine. The Hub and Hestia below both evaluate their law through it; it is published on crates.io only, with no wheel or browser build. Among the first two, the one unfilled cell is a browser build of `web4-core`, which has no WASM target in-tree; because a package's registry name is not always its crate name, resolve the name on the registry before depending on it.

The reference SDK — pure Python, offline, test-vector-driven. Alongside the Rust core, `web4-standard/implementation/sdk` (PyPI: `web4-sdk`) implements the standard in dependency-free Python: LCTs, T3/V3, **MRH graphs** (the relevancy horizon's reference home — the one core primitive not yet ported to the Rust crate), ATP, R6/R7, federation data structures, and capabilities. It is validated directly against the standard's conformance vectors, which makes it the executable reading of the specs: when prose and code disagree, the vectors arbitrate.

The Hub — a running Web4 society. `web4/hub` is a live society implementation: LCT-pinned membership, sealed member-to-member channels, a witnessed hash-chained ledger as the society's collective memory, law published as inspectable data gating actions, and role assignment through governance. It is where the SAL pattern, the witnessing fabric, and the membrane security model run as a daemon rather than a diagram — and the project runs it daily for its own multi-agent coordination. Its own README scopes that status precisely: the public tree is a **reference proof-of-concept** (`0.1.0-alpha.0`; MVP complete and pilot-ready, with the first community-chapter deployment still ahead), and production development continues in a separate private repository. The daily self-operation is the checkable part; "production" would be a label.

Hestia — agent governance at the membrane. `hestia` implements the trust architecture at the individual-agent boundary: policy evaluation gating agent tool use (the MCP membrane made enforceable), role-scoped law for autonomous sessions, and a witnessed record of every governed decision. It is the reference for Web4's answer to the question this paper opened with — how an agent is bounded *before* it acts.

It is also the reference for how to state what a mechanism does *not* do. Its maintainers publish the gate's limits as a first-class artifact (hestia#49 — preserved as evidence and superseded for tracking by the open readiness index hestia#224 — and its bypass catalogue): the gate stops accidents rather than adversaries; its posture when the policy daemon returns no verdict is fail-open unless a deployment explicitly configures it closed; the record covers *governed* activity, so silence in it is not evidence that nothing happened. Those limits are this paper's own thesis turned on the implementation — a gate that **declares** itself safe hands the relying party no evidence. What a relying party can compute on is the witnessed record and the named gaps, which is precisely why they are named.

Simulations — the adversarial gym. The `simulations/` suite exercises the mechanisms at scale against synthetic actors: trust networks, decay and economics, federation lifecycle, metabolic states — and an attack catalog of several hundred vectors across dozens of tracks, with roughly

85% detection. The number deserves its caveat in the paper, not just the repo: those adversaries are *synthetic*. The suite demonstrates that the defenses fire against the attacks anyone thought to write down; it is not a red-team result, and no live federation has yet been attacked by reality. Relatedly, inter-society federation is **specified but not yet built between live hubs** — the protocol documents exist, the SDK carries the data structures, and the first real society-to-society link is still ahead. This paper's honesty rule applies to the project itself: presence, tensors, grammar, and hub run today; live federation and a live ATP economy are the open frontier.

These are research-stage implementations, offered as existence proofs and starting points — not finished products. They are also the standard's proving ground: several normative requirements (fail-closed evaluation on invalid law, role-scoped trust, law-as-data) were hardened *by* operating these systems and folding what broke back into the specification.

Ecosystem bridges

Web4 is designed to compose with adjacent standards rather than replace them: the `did:web4` method bridges LCTs to W3C Decentralized Identifiers, credential formats align with the verifiable-credentials world, and the RDF backbone speaks the existing semantic web. The intent is an internet upgraded in place, not a parallel one.

Conclusion

This paper opened with two questions no current architecture answers cleanly: how to know an agent will act appropriately *before* it acts, and how to prove what it did *after*, without a single trusted referee. The Web4 answer is one equation:

```
Web4 = MCP + RDF + LCT + T3/V3 * MRH + ATP/ADP
```

Read back with its meaning now unpacked: entities interact through a **trust-aware membrane** (MCP), speaking statements with **verifiable meaning** (RDF), as **presences that cannot be transferred or faked** (LCT), trusted **for what their witnessed record shows** (T3/V3), **within the contexts where that record applies** (MRH), in a system where **allocation flows back to recognized contribution** (ATP/ADP). On those six primitives, the standard composes an action grammar (R6/R7), roles with their own presence, self-governing societies with inspectable law, and semantic bridges between domains.

The claims are testable and the artifacts are public. The standard is versioned in the open; the core packages, the Hub society, and Hestia run today as research-stage reference implementations. Status is marked honestly throughout: some mechanisms are v1.0 specifications with shipped code, others are drafts, and the value cycle — now shipped as a reference implementation — still awaits its first live economy. This is research in progress, developed in the open, and evaluable on its own terms.

Legal and organizational framework

The LCT mechanism is protected by two issued U.S. patents — US11477027 and US12278913 — with additional filings pending, held to keep the foundational mechanisms open for public benefit. The published implementations are **AGPL-3.0-or-later**; the patent rights granted by AGPL-3.0 §11 are not restricted to non-commercial use. See PATENTS.md for scope and separate licensing options. Development is supported by **MetaLINXX Inc.** The conceptual layer draws on the

Synchronism research program; Web4 itself is practical architecture, evaluable as protocols and running code.

An invitation

Web4 is not a product to purchase or a platform to join — it is a standard to challenge and build on. Read the specs. Run the packages. File the issue that breaks our assumptions. Engagement at any depth, from running code to disputing a normative requirement, is welcome at the project repository.

Trust, made native to the internet, changes what the internet can hold.

Glossary

Compact definitions for every term this paper relies on. The standard's GLOSSARY.md is the authoritative, complete vocabulary.

The equation elements

MCP — Model Context Protocol. The open protocol through which agents discover and invoke tools, read resources, and exchange context. Web4's I/O membrane: every boundary crossing is attributable, contextual, and witnessed.

RDF — Resource Description Framework. The W3C standard for typed subject–predicate–object statements. Web4's ontological backbone: every relationship (trust, witnessing, delegation, relevance) is an RDF triple, extensible without central coordination.

LCT — Linked Context Token. Web4's presence primitive: a non-transferable, cryptographically anchored record permanently bound to one entity, accumulating witnessed history over its lifecycle (genesis → active → rotation → void/slashed). Every LCT *contains* six required components — identity, binding, MRH, policy, T3, V3 — so each presence carries its own horizon and tensors. The foundation every other mechanism builds on.

LCT capability levels. The six-grade ladder of presence maturity: STUB, MINIMAL, BASIC, STANDARD, FULL, HARDWARE. Levels 0–4 accrete with the record; level 5 (hardware anchor) requires re-issuance, since the anchor must sit inside the signed binding from creation.

Device constellation. A Root LCT plus its Device LCTs, each bound to one hardware anchor and cross-witnessing the others. Trust is capped by anchor composition (single software key 0.40 → three-plus diverse hardware 0.98); recovery follows a quorum of the constellation. Identity as coherence across witnesses.

T3 — Trust Tensor. Capability measured along three root dimensions — **T**alent, **T**raining, **T**emperament — each an open-ended RDF sub-graph of context-specific sub-dimensions. Always bound to an entity-*role* pair, never to an entity alone.

V3 — Value Tensor. Contribution measured along three root dimensions — **V**aluation, **V**eracity, **V**alidity — same fractal RDF pattern. The verification side of `T3/V3`: claimed capability checked against delivered value.

MRH — Markov Relevancy Horizon. An entity's zone of relevance — what it can perceive, act on, and be affected by — implemented as a typed RDF relationship graph inside the LCT, with

three canonical edge kinds (`boundTo`, `pairedWith`, `witnessedBy`) and bounded traversal (default depth 3). The $*$ in the equation: all trust is contextualized by horizon.

ATP/ADP — Allocation Transfer / Discharge Packets. The value-feedback cycle, modeled on cellular energy metabolism: work discharges ATP into ADP; witnessed, recognized contribution recharges ADP into ATP. Semi-fungible (units equivalent, histories distinct). A feedback layer on the foundation — not a foundation.

Composed concepts

Entity. Anything that can bear presence — human, AI agent, organization, role, task, resource, or device. Defined by having (or being able to have) an LCT.

R6. The anatomy of every Web4 action: **R**ules + **R**ole + **R**equest + **R**eference + **R**esource → **R**esult.

R7. R6 with **R**eputation as a first-class output: the Request↔Result delta feeds back into the actor's T3/V3 tensors. The mechanism by which trust is computed rather than declared.

Role (as entity). A capacity — "Data Analyst," "Citizen," "Witness" — holding its own LCT, permission boundaries, and performance history, independent of whoever currently performs it. Agents pair with roles; both records update.

Base-mandatory roles. The eight roles every Web4 society must have filled — Sovereign, Law Oracle, Policy-Entity, Treasurer, Administrator, Archivist, Citizen, Maintainer — with one entity free to hold several. The **Maintainer** examines its entity's own standing state on a declared cadence and records each finding with a clearing condition; its MRH bounds what it examines, its entity's law bounds what it may repair. Any entity with an MRH may define one.

Witnessing. The corroboration fabric: entities observing and signing records of other entities' acts, making presence and history tamper-evident through accumulated independent observation rather than central authority.

Society. A bounded group of LCT-bearing members with shared law and collective memory — itself an entity with presence. Societies interact through the same MCP membrane as individuals (fractal governance).

SAL — Society–Authority–Law. The governance pattern: society as bounded membership, authority as explicitly delegated (never assumed) power, law as inspectable versioned *data* gating actions through the R6 Rules slot.

AGY — Agency delegation. The pattern for explicit, revocable grants of authority from one entity to another, every grant an inspectable link in the relationship graph.

ACP — Agentic Context Protocol. The extension of MCP for agents that *initiate*: plan → approve → execute → record, keeping autonomous action inside the accountability grammar.

Dictionary entity. A living, LCT-bearing translator between domain vocabularies, accumulating its own trust record as it translates — making semantic fidelity witnessed and measurable.

Compression-trust. The principle that communication efficiency depends on shared context: dense meaning transfers safely only where trust in shared understanding is high; dictionary entities manage the decompression across boundaries.

did:web4. The DID method bridging LCTs to the W3C Decentralized Identifier ecosystem.

References

Primary Sources

[1] Palatov, D. et al. (2024). "What is Web4 and Why Does It Matter." MetaLINXX Inc. Foundation Document.

[2] Palatov, D. (2025). "LCT_T3_ATP Integration with Anthropic Protocol - Entity Types and Roles." Web4 Technical Specification.

[3] Palatov, D. (2025). "Role-Entity LCT Framework." Web4 Architecture Document.

[4] MetaLINXX Inc. (2024). "Coherence Ethics: Synchronism and Emergent Systems." Philosophical Framework.

Patents

[5] Palatov, D. US Patent 11,477,027 B1: "Apparatus and Methods for Management of Controlled Objects." United States Patent and Trademark Office.

[6] Palatov, D. US Patent 12,278,913 B2: "Apparatus and Methods for Management of Controlled Objects." United States Patent and Trademark Office. (Continuation-in-part of the application issued as US 11,477,027.)

Technical Implementations

[7] Palatov, D. (2025). "Fractal Lightchain Architecture." Web4 Architecture Document (source repository not public).

[8] Palatov, D. (2025). "SAGE: Situation-Aware Governance Engine." https://github.com/dp-web4/HRM

[9] Palatov, D. (2025). "AI-DNA Discovery: Coherence Engine Implementation." https://github.com/dp-web4/ai-dna-discovery

[10] Palatov, D. (2025). "Web4 Cognition Pool." https://github.com/dp-web4/web4

Related Work

[11] Sapient Inc. (2024). "Hierarchical Reasoning Model (HRM)." https://github.com/sapientinc/HRM

[12] Aragon, R. (2024). "Transformer-Sidecar: Bolt-On Persistent State Space Memory." https://github.com/RichardAragon/Transformer-Sidecar-Bolt-On-Persistent-State-Space-Memory

[13] Model Context Protocol Specification. https://github.com/modelcontextprotocol/modelcontextprotocol

Theoretical Foundations

[14] Shannon, C. E. (1948). "A Mathematical Theory of Communication." Bell System Technical Journal.

[15] Von Neumann, J. (1958). "The Computer and the Brain." Yale University Press.

[16] Hofstadter, D. R. (1979). "Gödel, Escher, Bach: An Eternal Golden Braid." Basic Books.

[17] Kahneman, D. (2011). "Thinking, Fast and Slow." Farrar, Straus and Giroux.

Blockchain and Distributed Systems

[18] Nakamoto, S. (2008). "Bitcoin: A Peer-to-Peer Electronic Cash System."

[19] Buterin, V. (2014). "Ethereum: A Next-Generation Smart Contract and Decentralized Application Platform."

[20] Lamport, L. (1998). "The Part-Time Parliament." ACM Transactions on Computer Systems.

[21] Castro, M., & Liskov, B. (1999). "Practical Byzantine Fault Tolerance." OSDI.

Memory and Cognition

[22] Tulving, E. (1985). "Memory and Consciousness." Canadian Psychology.

[23] Hassabis, D., Kumaran, D., Summerfield, C., & Botvinick, M. (2017). "Neuroscience-Inspired Artificial Intelligence." Neuron.

[24] Graves, A., Wayne, G., & Danihelka, I. (2014). "Neural Turing Machines." arXiv preprint.

[25] Vaswani, A., et al. (2017). "Attention Is All You Need." NeurIPS.

Trust and Reputation Systems

[26] Resnick, P., & Zeckhauser, R. (2002). "Trust Among Strangers in Internet Transactions." The Economics of the Internet and E-commerce.

[27] Josang, A., Ismail, R., & Boyd, C. (2007). "A Survey of Trust and Reputation Systems for Online Service Provision." Decision Support Systems.

Complex Systems and Emergence

[28] Holland, J. H. (1995). "Hidden Order: How Adaptation Builds Complexity." Perseus Books.

[29] Wolfram, S. (2002). "A New Kind of Science." Wolfram Media.

[30] Mitchell, M. (2009). "Complexity: A Guided Tour." Oxford University Press.

Collaborative Intelligence

[31] Malone, T. W. (2018). "Superminds: The Surprising Power of People and Computers Thinking Together." Little, Brown and Company.

[32] Woolley, A. W., et al. (2010). "Evidence for a Collective Intelligence Factor in the Performance of Human Groups." Science.

Web Evolution

[33] Berners-Lee, T., Hendler, J., & Lassila, O. (2001). "The Semantic Web." Scientific American.

[34] O'Reilly, T. (2005). "What Is Web 2.0: Design Patterns and Business Models for the Next Generation of Software."

[35] Wood, G. (2014). "Ethereum: A Secure Decentralised Generalised Transaction Ledger."

Additional Resources

Websites

- Web4 Project: https://metalinxx.io/web4
- Web4 GitHub: https://github.com/dp-web4
- MetaLINXX Inc.: https://metalinxx.io

Contact

- Metalinxx Inc.: via the project repository
- Web4 Development: web4@metalinxx.io

Contributing

To contribute to Web4 development or request access to additional technical documents, please contact the development team through the channels above.

This reference list will be updated as the Web4 framework evolves and new implementations are developed.

Generated: 2026-11-06 04:32:21